day02pm

Introduction to Pointers

Kittipitch Kuptavanich

2025-03-28

Grader

- **C2P10: Insert in Order**
- **C2P11: Insert at Index**
- **C2P73: Reverse Range**
- **C2P92: Zoo Tour**

The swap() Function

- ต้องการเขียนฟังก์ชัน swap เพื่อใช้สลับค่าระหว่าง integer 2 ตัว

```
int main() {
    int x = 7, y = 8;
    printf("before: x = %d, y = %d\n", x, y);
    swap1(x, y);
    printf("after1: x = %d, y = %d\n", x, y);
    return 0;
}

void swap1(int x, int y) {
    int temp = x;
    x = y;
    y = temp;
}
```

clone clone

```
before: x = 7, y = 8
after1: x = 7, y = 8
```

The swap() Function [2]

- แก้ปัญหาโดยการส่งที่อยู่ (memory address) เข้าไปแทน แล้วเรียกอ่าน/เขียนค่าผ่านที่อยู่นั้น ๆ

```
void swap2(address x_add, address y_add) {

        int temp = value in x_add;
  value in x_add   = value in y_add;
  value in y_add   =        temp;
}
```

```
void swap2(int   *x_add, int   *y_add) {

        int temp =        *x_add;
        *x_add   =        *y_add;
        *y_add   =        temp;
}
```

The swap() Function [3]

- **Pointer declaration and usage**

```
void swap2(int *x_add, int *y_add) {

    int temp = *x_add;
    *x_add   = *y_add;
    *y_add   = temp;
}
```

- ประกาศ
  - `<type>* ptr;`
- เรียกใช้
  - `ptr` คือที่อยู่ (address) ของตำแหน่งในหน่วยความจำ
    มีชนิดข้อมูลเป็น `<type> *` เช่น `char *` หรือ `int *`
  - `*ptr` คือค่าที่เก็บภายในของ `ptr`

The swap() Function [4]

- **call by reference คือการเรียกใช้ function โดยการส่งข้อมูล address ของ พารามิเตอร์ (input) เข้าไป**

```
05    void swap2(int *x_add, int *y_add) {
06        int temp = *x_add;
07        *x_add   = *y_add;
08        *y_add   =  temp;
09    }
10
11    int main() {
12        int x = 7, y = 8;
13        printf("before x = %d, y = %d\n", x, y);
14        swap2(&x, &y); // &x and &y= address of x and y
15        printf("after2 x = %d, y = %d\n", x, y);
16        return 0;
17    }
```

```
before x = 7, y = 8
after2 x = 8, y = 7
```

The swap() Function [5]

- ใน C++ เราสามารถส่ง Argument แบบ pass-by-reference โดยไม่ต้องใช้ pointer

```
void swap(int &a, int &b) {
    int temp = a;
    a = b;
    b = temp;
}

int main() {
    int x = 7, y = 8;
    std::cout << "before x = " << x << ", y = " << y << std::endl;
    swap(x, y);
    std::cout << "after3 x = " << x << ", y = " << y << std::endl;
    return 0;
}
```

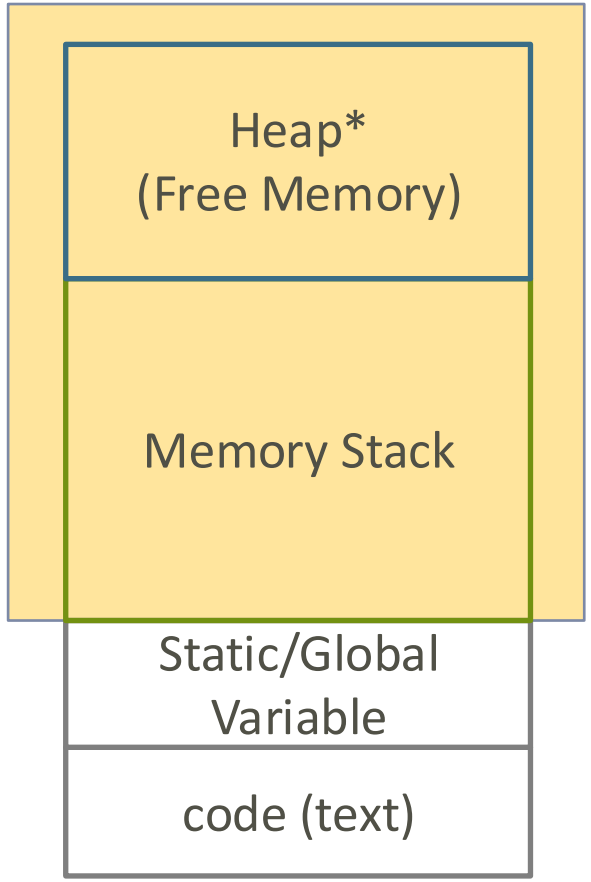
```
before x = 7, y = 8
after3 x = 8, y = 7
```

Pointers and Addresses

- **Pointer: ตั้งชื่อให้ memory address ของ variable**
- **เราสามารถอ่าน/เขียนค่าของตัวแปรผ่าน Address ได้จากทุกที่**
- **32 bit OS: address ทั้งหมด 2^{32} ตำแหน่ง**
  - $2^{32} = 4 * 1024 * 1024 * 1024$
  - **RAM ไม่เกิน 4GB**
- **64 bit OS: address ทั้งหมด 2^{64} ตำแหน่ง**
  - **RAM ได้ถึง 16 exabyte = 17,179,869,184GB**

Application Memory Allocation

- ในการรันโปรแกรม จะมีการจัดสรรหน่วยความจำ 2 รูปแบบคือ **Stack** และ **Heap**
- **Stack** จัดการโดย **OS**
  - เก็บค่าตัวแปร แต่ละฟังก์ชัน
  - <u>คืนพื้นที่อัตโนมัติ</u> หลังจบฟังก์ชัน
- **Heap** จัดการจองและคืนโดยโปรแกรมเมอร์ ผ่านคำสั่ง `malloc()`, `calloc()`, `realloc()` และ `free()`



A First Book of ANSI C, 4th Edition

Dynamic Memory Allocation [2]

- **Stack**
  - ข้อมูลที่มีขนาดเล็ก ใช้เฉพาะในฟังก์ชันใดฟังก์ชั่นหนึ่ง
  - จัดการให้อัตโนมัติ ไม่จำเป็นต้องเขียนคำสั่งเพิ่มเติม
- **Heap** เหมาะกับ
  - การเก็บข้อมูล **array** หรือ `struct` ขนาดใหญ่ เช่น `int arr[1000000];`
  - ตัวแปรที่ต้องการเก็บไว้ใช้ร่วมกันหลายฟังก์ชัน
  - **array** หรือ **struct** ที่มีการเพิ่มหรือลดขนาดได้
  - โครงสร้างข้อมูลแบบต่าง ๆ **(Linked List, Tree, Graph, etc.)**
  - **Programmer** จัดการการใช้งานเอง

A First Book of ANSI C, 4th Edition

Pointers and Arrays

- ชื่อของ array (or the value it holds) จริง ๆ แล้วคือ constant pointer ที่ชี้ไปยัง element แรกของ array นั้น ๆ

```
int *pa = arr; ⇔ int *pa = &arr[0];
```

- A + i คือ address ของ A[i] = &A[i]
- printf("%p", &A[0]);
- Defining an array
  - int A[10] // static array of 10 int's
  - char* A[10] // static array of 10 char *'s
  - int* A[10] // static array of 10 int *'s

Pointers and Arrays [2]

- เนื่องจาก array เป็นการอ้างถึง address บน memory อยู่แล้ว ดังนั้น การเรียกใช้ function โดยมี array เป็นพารามิเตอร์ ก็จะเป็นการ call by reference เช่นกัน

```
void intswap(int A[], int i, int j) {
  int temp = A[i];
  A[i] = A[j];
  A[j] = temp;
}

```

Pointer Arithmetic

- `my_strcpy()` **example**

```
05  void my_strcpy(char dest[], char src[]) {
06    char *dp = &dest[0], *sp = &src[0];
07    while(*sp != '\0')
08      *dp++ = *sp++;
09    *dp = '\0';
10  }
```

```
07    while(*sp != '\0') {
08      *dp = *sp;
09      dp++;
10      sp++;
11    }
```

- `*dp++` มีการทำงานเหมือน `*(dp++)`
- ควรต้องตรวจสอบแน่ใจก่อนว่า array ปลายทางมีขนาดใหญ่พอ

FUNCTION POINTERS

Function Pointers

- พิจารณา Code ด้านล่าง: 3 ฟังก์ชัน ที่ทำหน้าที่แตกต่างกัน

```
int add(int a, int b) {
    return a + b;
}

int subtract(int a, int b) {
    return a - b;
}

int multiply(int a, int b) {
    return a * b;
}
```

Function Pointers [2]

- เราสามารถเรียกใช้แต่ละฟังก์ชันด้วย **Code** เดิม แต่มี ผลลัพธ์ต่างกัน

```
05  int add(int a, int b) {
06      return a + b;
07  }
08
09  int subtract(int a, int b) {
10      return a - b;
11  }
12
13  int multiply(int a, int b) {
14      return a * b;
15  }
```

```
15  int main()
16  {
17      int (*operation)(int, int);
18      int a = 10, b = 5;
19
20      operation = add;
21      cout << "+ " << operation(a, b) << endl;
22
23      operation = subtract;
24      cout << "- " << operation(a, b) << e
25
26      operation = multiply;
27      cout << "x " << operation(a, b) << endl;
28
29      return 0;
30  }
```

Function Pointer

Same Code

Function Pointers [3]

- **Function Pointer** คือตัวแปรที่เก็บที่อยู่ของฟังก์ชันในหน่วยความจำ
- การ **assign** ค่า
  - ไม่ใส่ & และใช้ชื่อฟังก์ชันได้ตรง ๆ

    เช่น `operation = add`

    ไม่ใช่ `operation = &add`

```
int main()
{
  int (*operation)(int, int);
  int a = 10, b = 5;

  operation = add;
  cout << "+ " << operation(a, b) << endl;

  operation = subtract;
  cout << "- " << operation(a, b) << endl;

  operation = multiply;
  cout << "x " << operation(a, b) << endl;

  return 0;
}
```

Function Pointer Declaration

- รูปแบบ:

```
return_type (*pointer_name)(argument_type1, argument_type2, ...);
```

- เช่น หากมีฟังก์ชัน *add* ที่รับสองจำนวนเต็มเข้ามาและคืนค่าผลบวกของทั้งสองจำนวน

```
void add(int a, int b);
```

- จะประกาศฟังก์ชันพอยน์เตอร์เพื่อที่จะใช้ชี้ไปยัง *add* ได้โดยใช้รูปแบบนี้:

```
int (*func_ptr)(int, int);
```

Function Pointers as Arguments

```
int main()
{
    double a = 10, b = 5;
    cout << "r area" << findArea(rectangleArea, a, b) << endl;
    cout << "t area" << findArea(triangleArea, a, b) << endl;

    return 0;
}
```

```
double rectangleArea(double a, double b)
{
    return a * b;
}

double triangleArea(double a, double b)
{
    return 0.5 * a * b;
}

double findArea(double (*fun)(double, double), double h, double w)
{
    return fun(h, w);
}
```

Function Pointer - Summary

```
bool equal(int x, int y)
{
    return x == y;
}

int main()
{
    bool (*fn)(int, int);
    bool ans;

    fn = equal;
    ans = fn(12, 30);
    cout << "ans = " << ans << endl;


    return 0;
}
```

- สร้าง function pointer ชื่อ fn
  - กำหนดว่า parameter คืออะไร
- ชี้ไปที่ ฟังก์ชัน equal()
- เรียกใช้

Lambda Functions

```
..
double rectangleArea(double a, double b)
{
  return a * b;
}

int main()
{
  double a = 10, b = 5;
  cout << "r area: " << findArea(rectangleArea, a, b) << endl;

  cout << "r area lambda: " \
       << findArea([](double a, double b) { return a * b; }, a, b) << endl;

  cout << "r area lambda 2: " \
       << findArea([](double a, double b) -> double{ return a * b; }, a, b) << endl;
  return 0;
}
```

Lambda Functions [2]

```
int x = 10;
int y = 20;

// Lambda with empty capture list
auto lambda1 = []() { return 42; };
cout << "Lambda 1: " << lambda1() << endl;

// Lambda capturing x and y by value
auto lambda2 = [x, y]() { return x + y; };
cout << "Lambda 2: " << lambda2() << endl;

// Lambda capturing x by reference
auto lambda3 = [&x]() { x += 10; };
lambda3();
cout << "Lambda 3: " << x << endl;

```

```
$ g++ -std=c++17 lambda_fun.cpp; ./a.out
Lambda 1: 42
Lambda 2: 30
Lambda 3: 20
```

Kittipitch Kuptavanich

STRUCTURES AND POINTERS

The swap() Function w/ Structs

```
struct point {
    double x;
    double y;
};
```

- ต้องการเขียนฟังก์ชัน swap เพื่อใช้สลับค่าระหว่าง จุด 2 จุด
  - สังเกตการใช้ -> กรณี pointer

```
void swapPtsWithRef(point& p1, point& p2)
{
    // Swap the x coordinates
    double tempX = p1.x;
    p1.x = p2.x;
    p2.x = tempX;

    // Swap the y coordinates
    double tempY = p1.y;
    p1.y = p2.y;
    p2.y = tempY;
}
```

```
void swapPtsWithPtr(point* p1, point* p2)
{
    // Swap the x coordinates
    double tempX = p1->x;
    p1->x = p2->x;
    p2->x = tempX;

    // Swap the y coordinates
    double tempY = p1->y;
    p1->y = p2->y;
    p2->y = tempY;
}
```

DYNAMIC DATA STRUCTURE: LINKED LIST

Static Data Structure

- พิจารณา การเก็บข้อมูลแบบ **Array**
  - มีขนาดจำกัด ต้องระบุไว้ล่วงหน้าว่า **Array** มีความจุเท่าไร
  - จำเป็นต้องจองพื้นที่เก็บข้อมูลที่ต่อเนื่องกันในหน่วยความจำเป็นชิ้นเดียว
  - มีความยุ่งยากในกรณีต่อไปนี้
    - กรณีที่เก็บข้อมูลจนเต็มพื้นที่
    - ต้องการลดขนาดเนื่องจากจองที่ไว้เกินกว่าที่ต้องการมาก

A First Book of ANSI C, 4th Edition

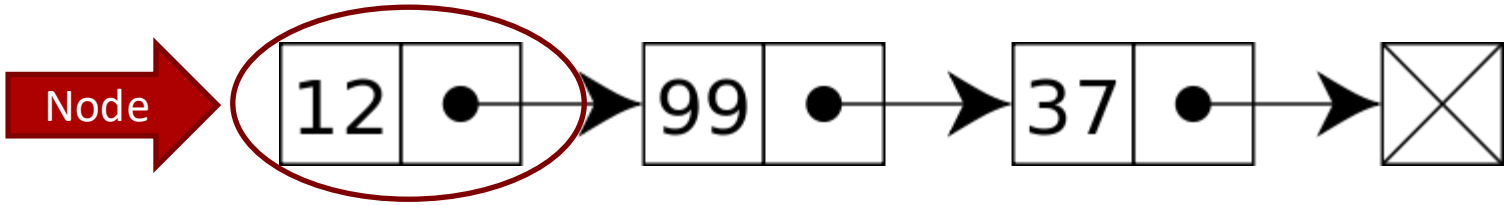
Dynamic Data Structure

- ไม่จำเป็นต้องมีการระบุจำนวนข้อมูลที่ต้องการเก็บ ไว้ล่วงหน้า
- จองที่ในหน่วยความจำ เมื่อมีการเพิ่มข้อมูล
  - จองหน่วยความจำแยกกันได้เป็นหน่วยย่อย ๆ
- ปล่อยหน่วยความจำที่ไม่ต้องการใช้ เมื่อมีการลบข้อมูล

A First Book of ANSI C, 4th Edition

Linked List

- **Definition: Linked List (ลิงท์ ลิสต์) เป็น Dynamic Data Structure ที่มีข้อมูล (record) เรียงต่อกัน โดยแต่ละ element มี link เชื่อมไปยัง record ถัดไป**
- **Linked List มีได้หลายประเภท เช่น Singly Linked, Doubly Linked, Circular – ใน class นี้เราจะพิจารณาแค่ Singly Linked List**

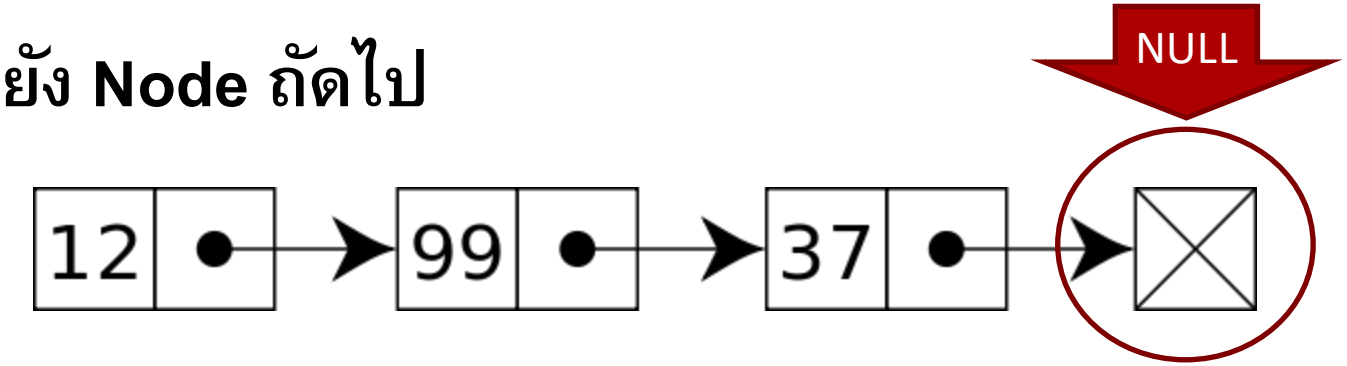


- **เรียกแต่ละ element ใน linked list ว่า Node**

A First Book of ANSI C, 4th Edition

Linked List [2]

- แต่ละ **Node** จะประกอบด้วย
  - ข้อมูลที่ต้องการเก็บ
  - **Link** ไปยัง **Node** ถัดไป

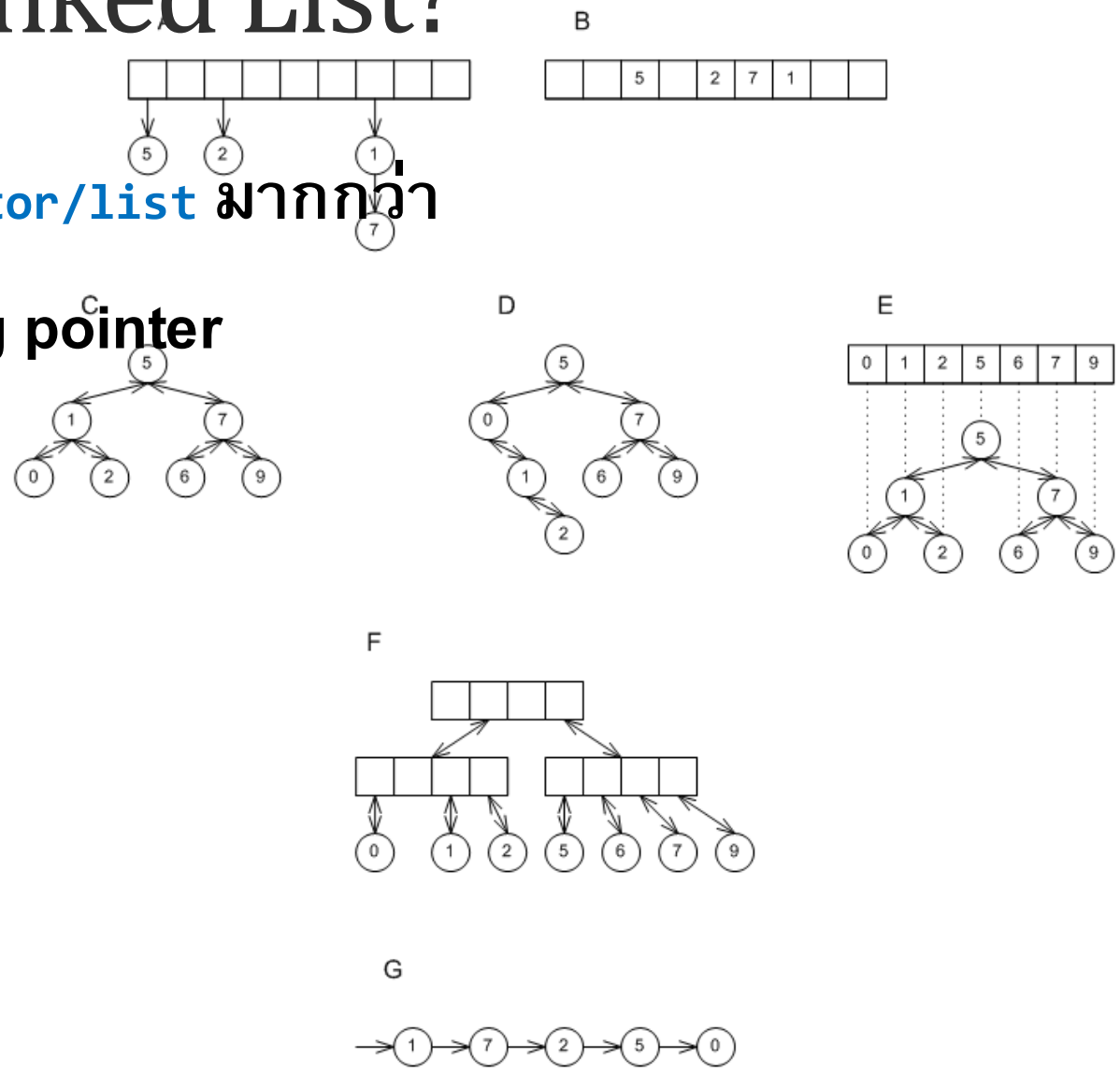


- ตำแหน่งท้ายสุดของ **List** จะมี **Link** ชี้ไปที่ **NULL**
- เราจะเข้าถึงข้อมูลใน **List** โดยการเก็บตำแหน่งแรกสุดของ **List (head)** ไว้

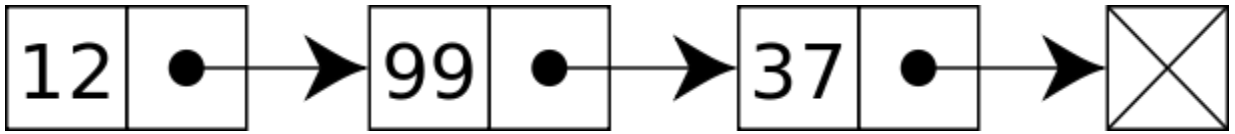
A First Book of ANSI C, 4th Edition

But wait… why Linked List?

- ปกติในการแข่งขันจะใช้ `vector/list` มากกว่า
  - ฝึกการ **manipulate/debug pointer**
- **Tree/Graph etc**



Syntax – Node Declaration




```
struct node
{
  int data;        /* can be any data type */
  struct node *next;
};
struct node *head;
```

ตำแหน่งหัว List (head)

```
/* สามารถใช้ typedef เพื่อสร้าง alias (shortcut) */
typedef struct node nodeT;
```

A First Book of ANSI C, 4th Edition

Dynamic Memory Allocation

- โดยปกติแล้ว ตัวแปรใด ๆ ที่สร้างขึ้นใน function จะไม่สามารถถูกเข้าถึง (เขียน/อ่าน) ได้ เมื่อจบการทำงาน function นั้น ๆ

```
pointT *createPoint(int x, int y) {
    pointT newPoint = {x, y};
    return &newPoint;
}
```

Will NOT Work!!

A First Book of ANSI C, 4th Edition

Dynamic Memory Allocation [2]

- หากต้องการสร้างตัวแปรที่มี data type เป็น struct ขึ้น โดยที่ต้องการให้มีการเข้าถึงข้อมูลในตัวแปรนั้น ๆ ได้ นอก function ที่สร้างตัวแปรนั้น ๆ
  - จำเป็นต้องมีการจองหน่วยความจำไว้อย่างถาวร โดยการใช้ฟังก์ชัน `malloc()`

```
void *malloc(size_t n)
```

- `malloc` = memory allocate
- Return pointer (ไม่ระบุชนิด – void *) ไปยังหน่วยความจำที่จองไว้ ตามขนาดที่ระบุ (n)
- หากจองหน่วยความจำไม่สำเร็จ จะ return NULL

A First Book of ANSI C, 4th Edition

Dynamic Memory Allocation [3]

```
nodeT *newNode = (nodeT *) malloc(sizeof(nodeT));
```

- เนื่องจาก `malloc()` มีการ return ค่าเป็น `void *` เราจำเป็นต้อง มีการเปลี่ยนชนิด (type casting) ของ pointer เป็นชนิดที่เราต้องการ
- ฟังก์ชัน `sizeof()` ใช้เพื่อวัดขนาดของ หน่วยความจำที่ต้องการจอง ในกรณีนี้เราต้องการขนาดเท่ากับขนาดของหนึ่ง `nodeT`
- หากต้องการจองที่สำหรับ `int` array ขนาด 10 หน่วย

```
malloc(sizeof(int) * 10)
```

A First Book of ANSI C, 4th Edition

Dynamic Memory Allocation [4]

```
void *calloc(size_t n, size_t size)
```

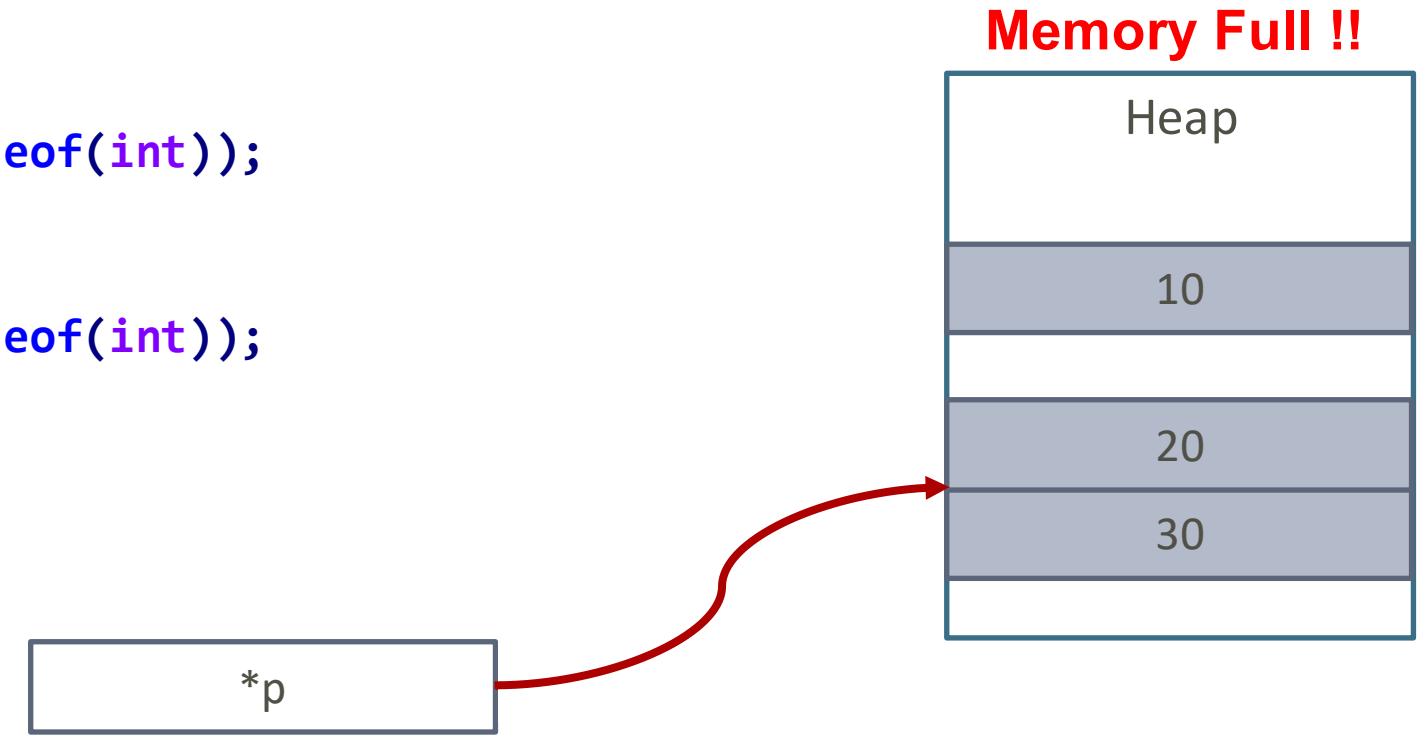
- จองพื้นที่หน่วยความจำในลักษณะ array ที่มีสมาชิก `n` ตัว โดยแต่ละตัวมีขนาด `size` byte
- พร้อมกำหนดให้ค่าเริ่มต้นทั้งหมดของสมาชิกแต่ละตัวเป็น 0

<u>ข้อควรระวัง:</u> หากมีการจองหน่วยความจำไว้อย่างถาวรเพิ่มขึ้นเรื่อย ๆ หน่วยความจำของคอมพิวเตอร์ก็จะเต็มในที่สุด

A First Book of ANSI C, 4th Edition

Dynamic Memory Allocation [5]

```
int main()
{
    int i;
    int *p= malloc(sizeof(int));
    *p=10;

    p= malloc(sizeof(int));
    *p=20;

    p= malloc(sizeof(int));
    *p=30;

}
```



A First Book of ANSI C, 4th Edition

Dynamic Memory Allocation [6]

ดังนั้นเมื่อไม่ใช้ตัวแปรใด ๆ ที่มีการจองหน่วยความจำไว้แบบถาวร เราจำเป็นต้องคืนหน่วยความจำนั้น ๆ โดยการใช้ฟังก์ชัน `free()`

```
void free(void *)
/* for example */
free(newNode)
```

- ใน application ใด ๆ จำนวน memory ที่จองด้วย `malloc()` จะต้องเท่ากับ memory ที่ `free()` ออก ในที่สุด เพื่อป้องกัน memory leak

A First Book of ANSI C, 4th Edition

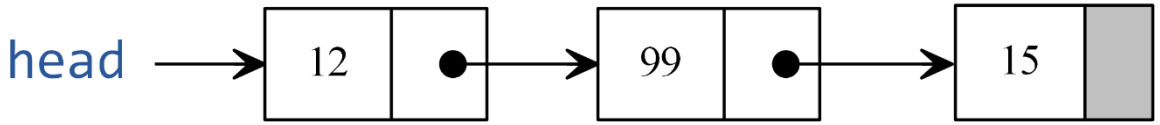
Creating a Node

```
nodeT *createNode(int data) {
  nodeT *newNode = (nodeT *) malloc(sizeof(nodeT));
  if (newNode == NULL)  /* not enough memory */
    return NULL;

  newNode->data = data; /*same as (*newNode).data*/
  newNode->next = NULL;
  return newNode;
}
```

A First Book of ANSI C, 4th Edition

Adding New Node to a List

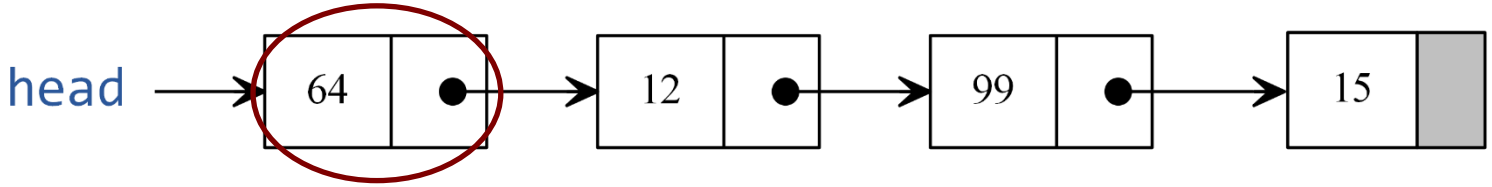
- เราสามารถเพิ่ม Node ใหม่เข้าไปที่ส่วนหัว (add to front) หรือส่วนท้าย (add to back)
- Original List



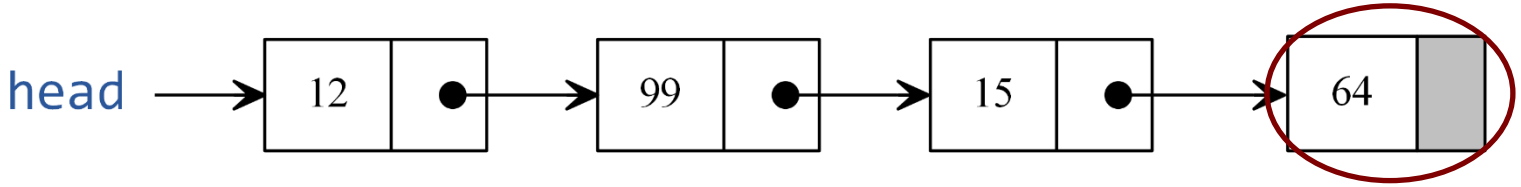
- newNode

- Add to Front

ถ้า head เป็น NULL
การ add newNode จะเป็นการสร้าง List ใหม่



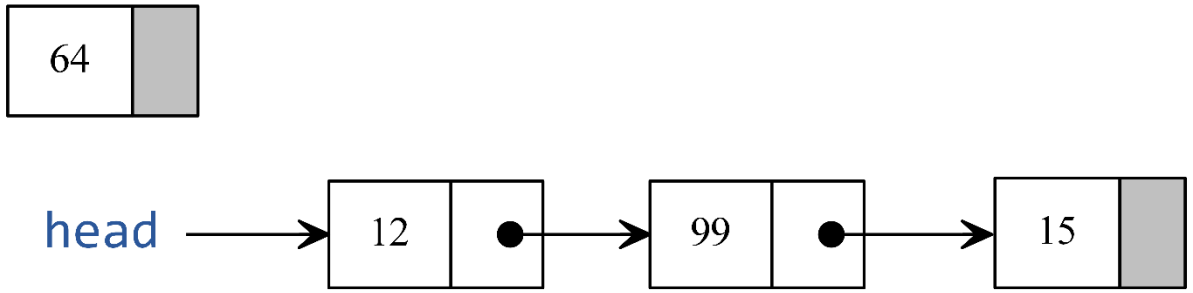
- Add to Back



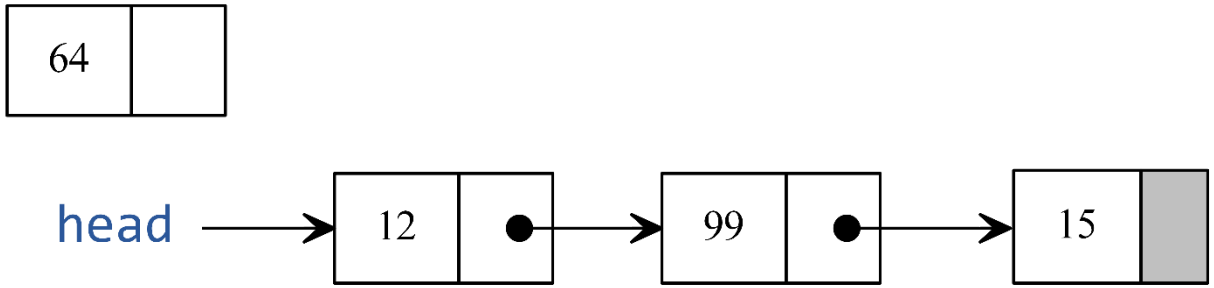
A First Book of ANSI C, 4th Edition

Add Node to Front

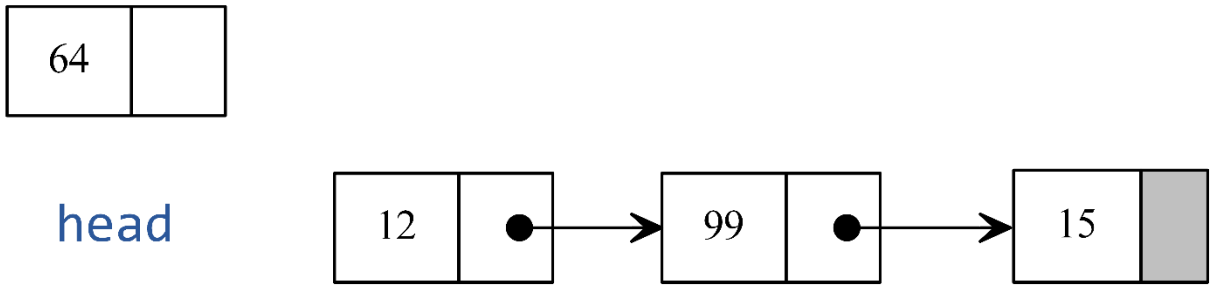
STEP 0:



STEP 1:



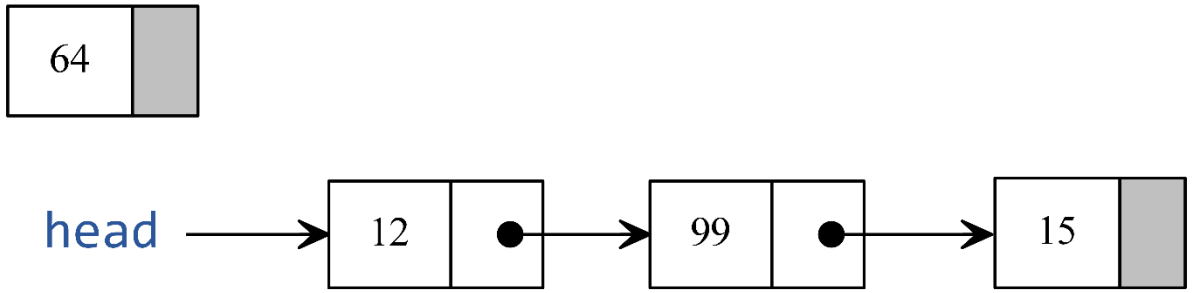
STEP2:



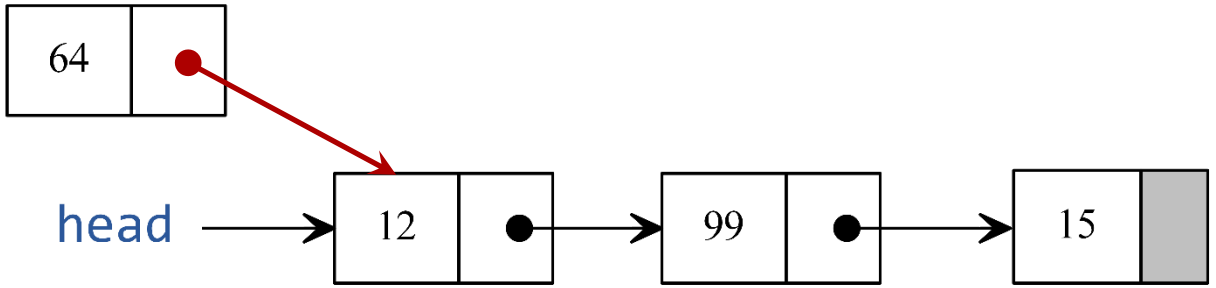
A First Book of ANSI C, 4th Edition

Add Node to Front

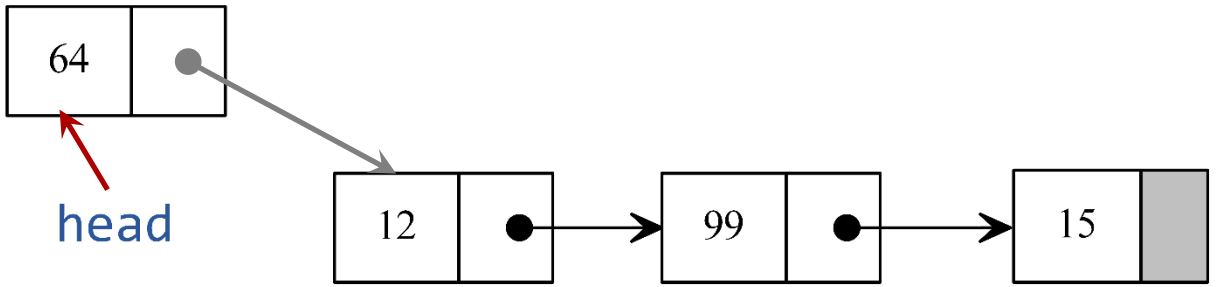
STEP 0:



STEP 1:



STEP2:



A First Book of ANSI C, 4th Edition

Add Node to Front [2]

```
nodeT *addToFront(nodeT *head, int data)
{
    nodeT *newNode = createNode(data);
    if (newNode == NULL) /* not enough memory */
        return head;
    newNode->next = head;
    return newNode;
}
```

ถ้าต้องการ **add to back** จะต้องทำอย่างไร?

A First Book of ANSI C, 4th Edition

List Iterating

- การเข้าถึงทีละสมาชิกสามารถทำได้โดย

```
for (p = head; p != NULL; p = p->next) {
  /* do  something */
}

for (p = head; p->next != NULL; p = p->next) {
  /* do  something */
}
```

A First Book of ANSI C, 4th Edition

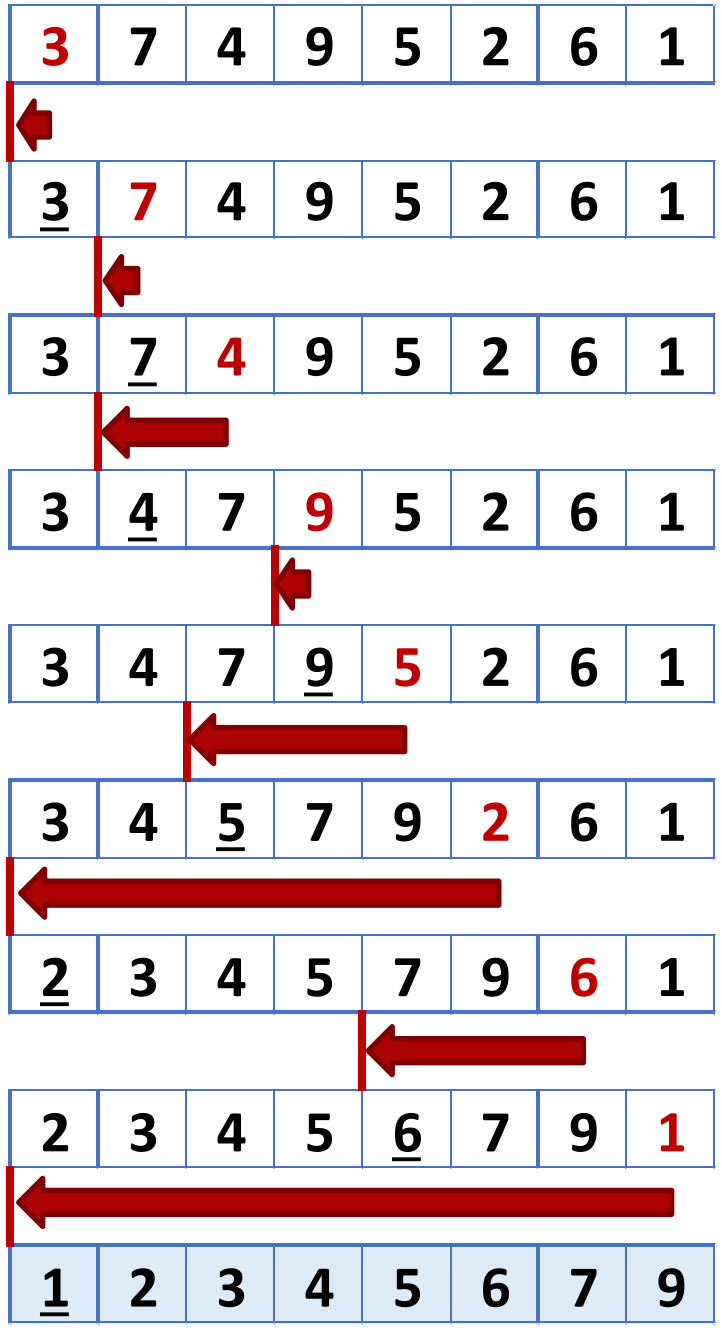
List Iterating [2]

- หรือใช้ while เช่น

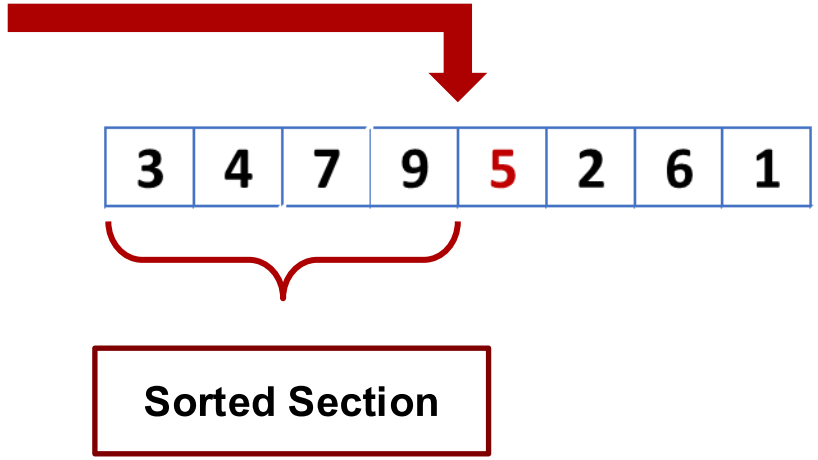
```
void printList(nodeT *head)
{

    nodeT *currNode = head;
    while (currNode != NULL) {

        printf("%d -> ", currNode->data);
        currNode = currNode->next;
    }
    printf("NULL\n");

}
```

A First Book of ANSI C, 4th Edition

Insertion Sort



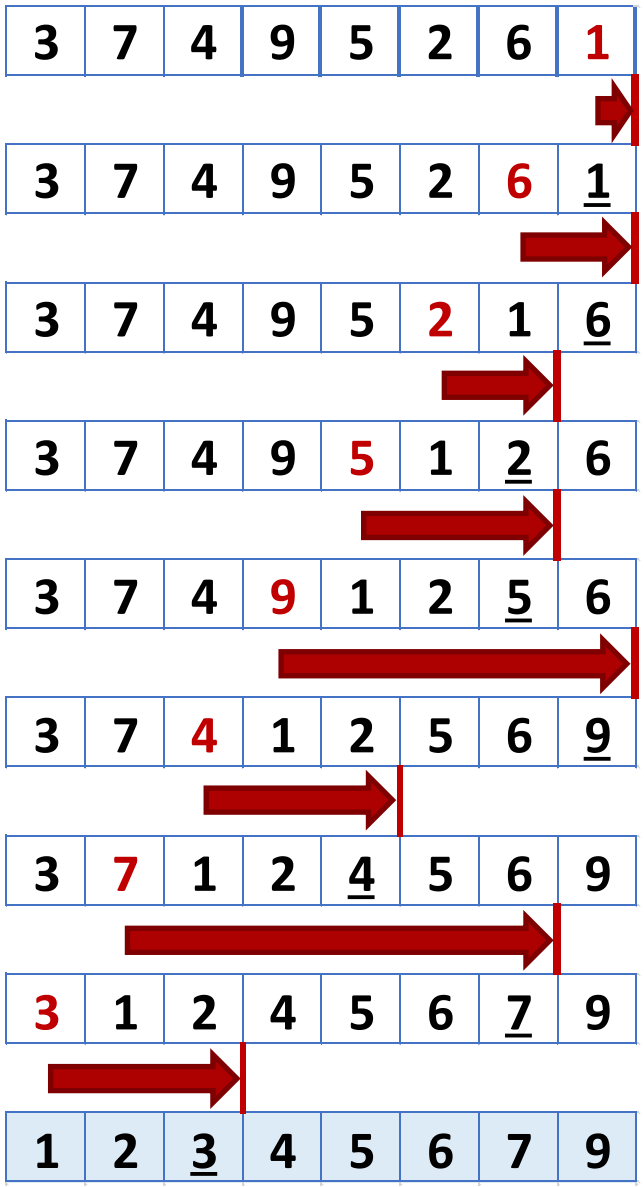
- เริ่มจากตำแหน่งซ้ายสุด
- จับวิ่งไปทางซ้ายจนพบค่าน้อยกว่า



Practice 1: Insertion Sort

6	5	3	1	8	7	2	4
	5	3	1	8	7	2	4
		3	1	8	7	2	4
			1	8	7	2	4
				8	7	2	4
					7	2	4
						2	4
							4

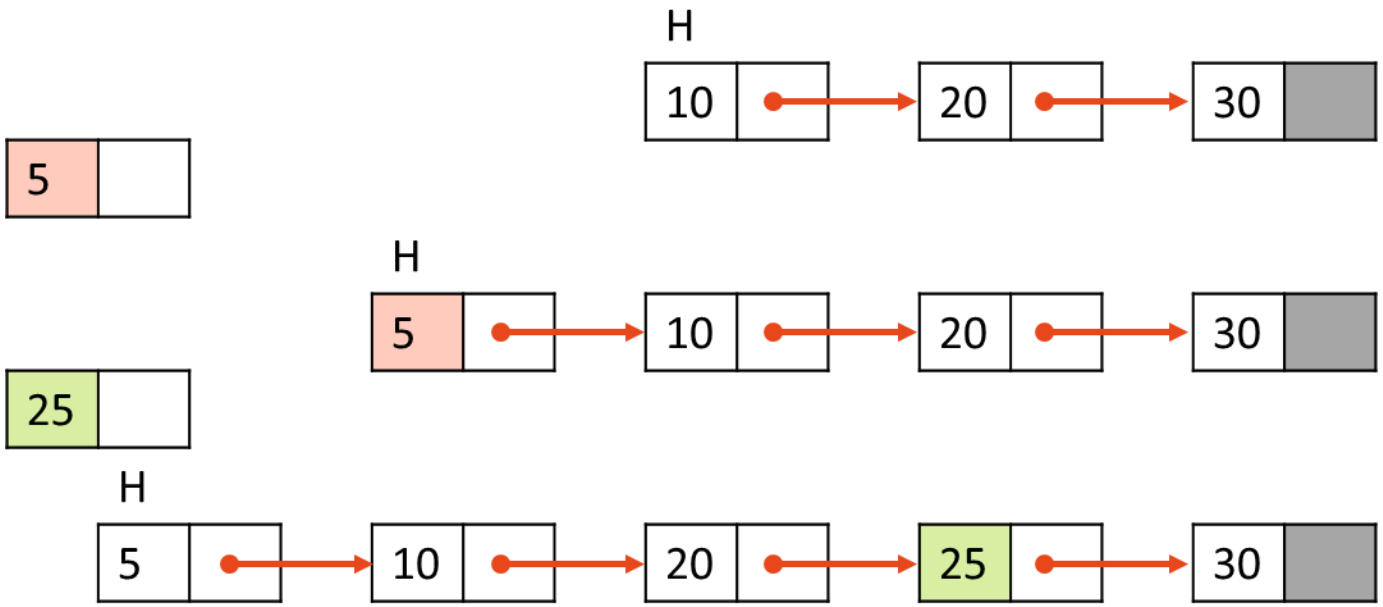
Insert in Order



- **Insertion Sort (Revisited)**
- ปรับอัลกอรึทึมให้เริ่มจากทางขวา
  - เริ่มจากตำแหน่งขวาสุด
  - จับวิ่งไปทางขวาจนพบค่ามากกว่า <u>จึงหยุด</u>

Insert in Order [2]

- ทำนองเดียวกันกับ insertion sort
  - จับ newNode วิ่งไปทางขวา (วิ่งจาก head ไปทางท้าย list) และหยุดเมื่อเจอ node ที่มีค่ามากกว่า



A First Book of ANSI C, 4th Edition

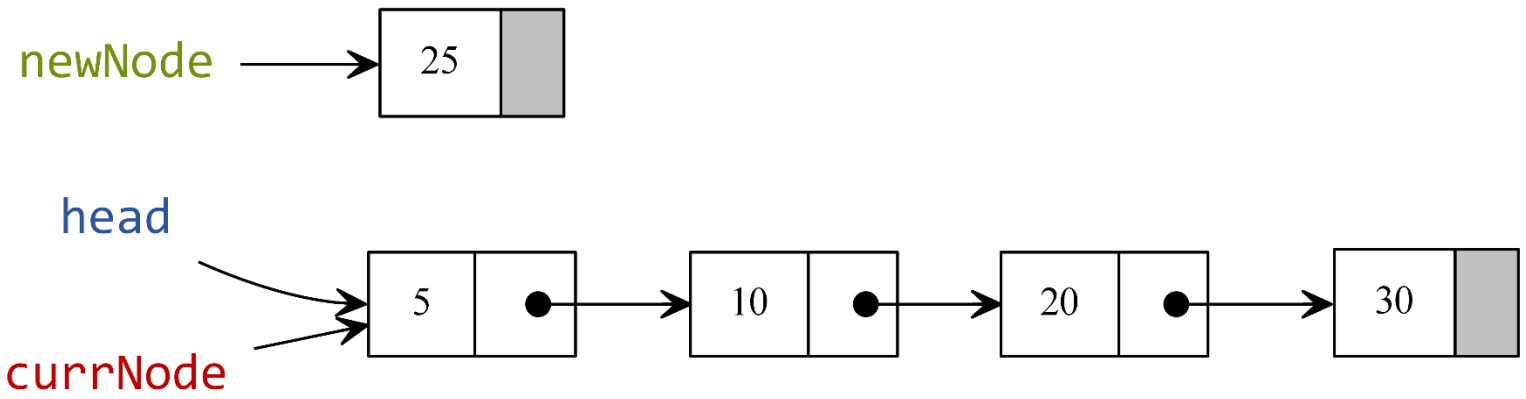
Insert in Order [3]

1. **Case 1: หาก `newNode->data` น้อยกว่า `head->data` หรือ `head == NULL`**
   - `return addToFront(head, data)`
2. **Case 2: วิ่งไล่ที่ ละ Node จนเจอ `nodeX` ที่ `nodeX->data > newNode->data`**
   - **นำ `newNode` ไว้ที่ตำแหน่งหน้า `nodeX` นั้น**

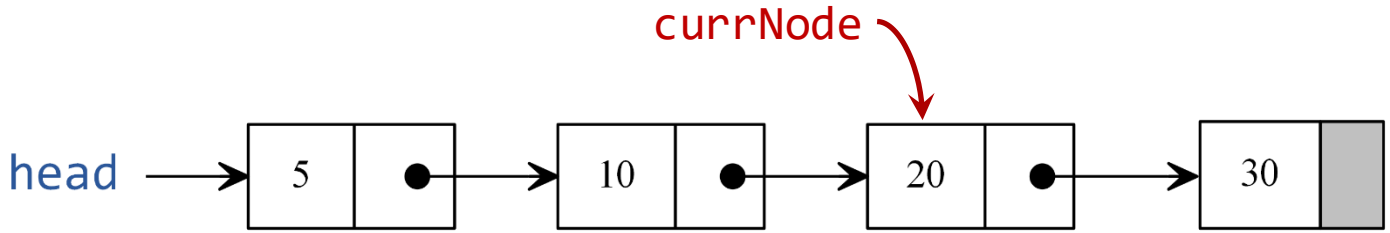
A First Book of ANSI C, 4th Edition

Insert in Order [4]

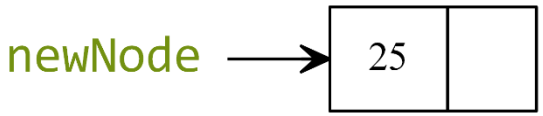
- **STEP 0:**



- **STEP 1:**



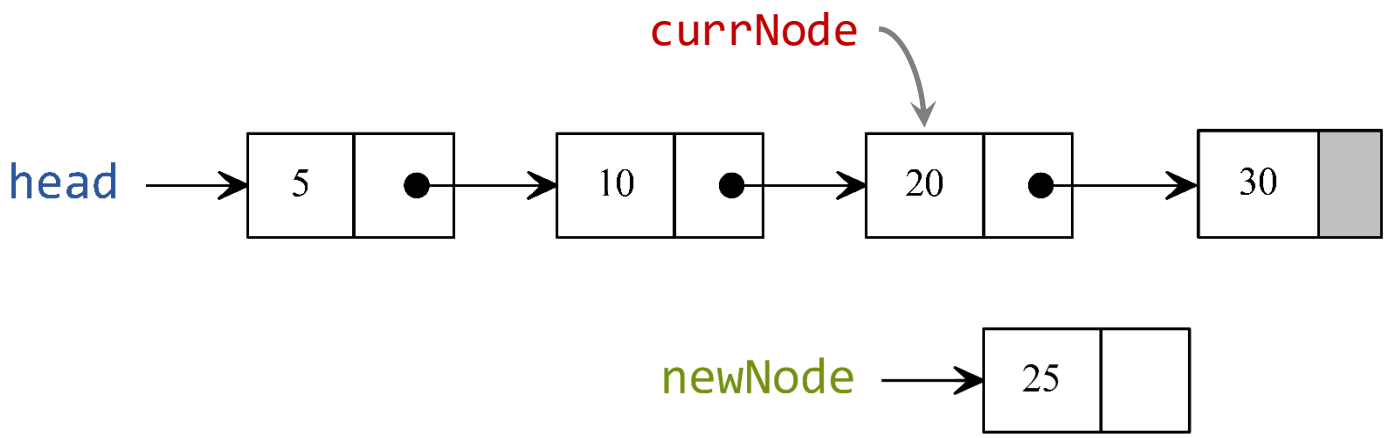

```
while ((currNode->next) &&
       (currNode->next->data < newNode->data))
       currNode = currNode->next;
```



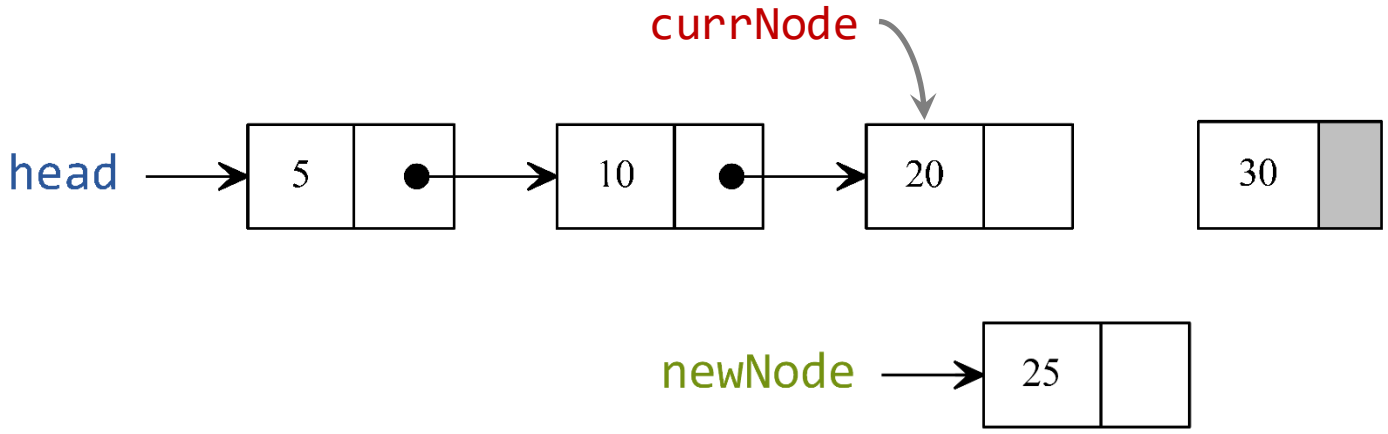
A First Book of ANSI C, 4th Edition

Insert in Order [5]

- **STEP 2:**



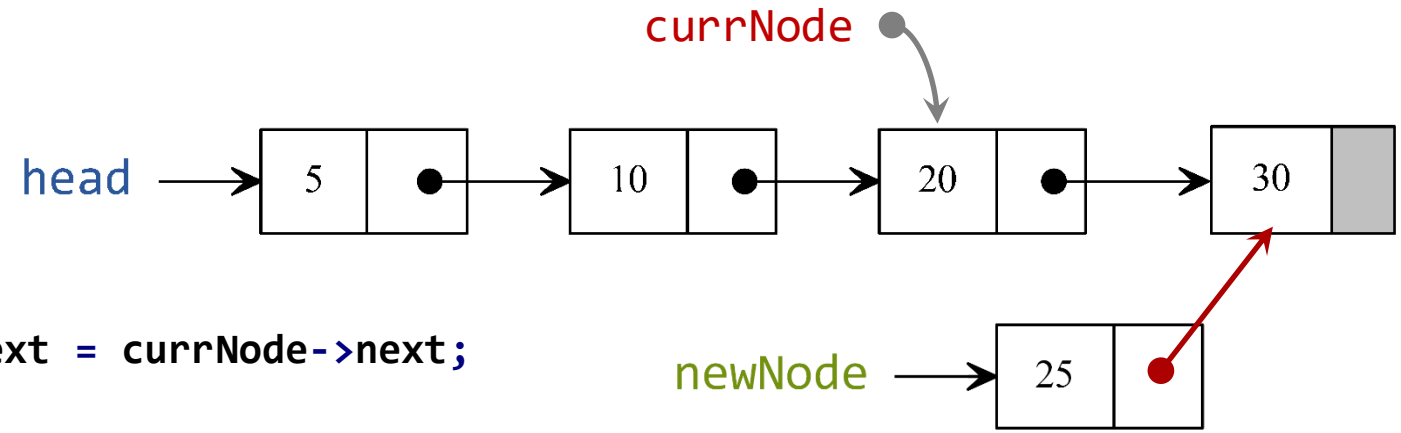
- **STEP 3:**



A First Book of ANSI C, 4th Edition

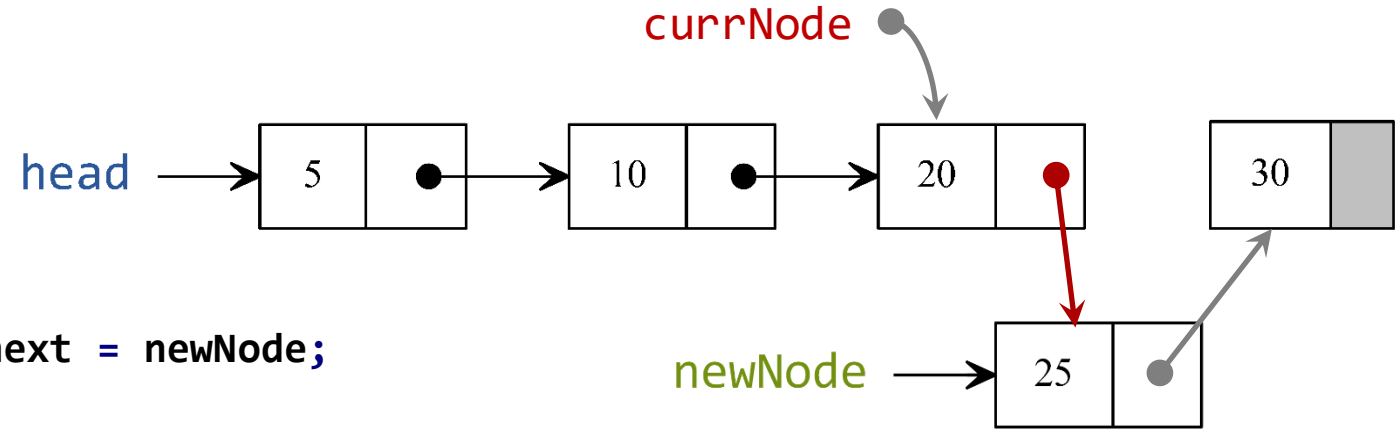
Insert in Order [5: Key]

- **STEP 2:**




```
newNode->next = currNode->next;
```

- **STEP 3:**




```
currNode->next = newNode;
```

A First Book of ANSI C, 4th Edition

Insert in Order [6: Key]

```
nodeT * insertInOrder(nodeT * head, int data)
{
    nodeT *currNode = head;
    nodeT *newNode = NULL;

    if (!head || data < head->data) {
        return addToFront(head, data);
    }
    newNode = createNode(data);

    if (newNode == NULL) /* not enough memory */
        return head;

    while ((currNode->next) &&
            (currNode->next->data < newNode->data))
        currNode = currNode->next;

    newNode->next = currNode->next;
    currNode->next = newNode;

    return head;
}
```

A First Book of ANSI C, 4th Edition

Find in List

```
nodeT *findInList(nodeT *head, int key)
{
    nodeT* currNode = head;
    while (currNode && currNode->data < key) {
        currNode = currNode->next;
    }
    if (                                )
        return          ;


}
```

A First Book of ANSI C, 4th Edition

Find in List [Key]

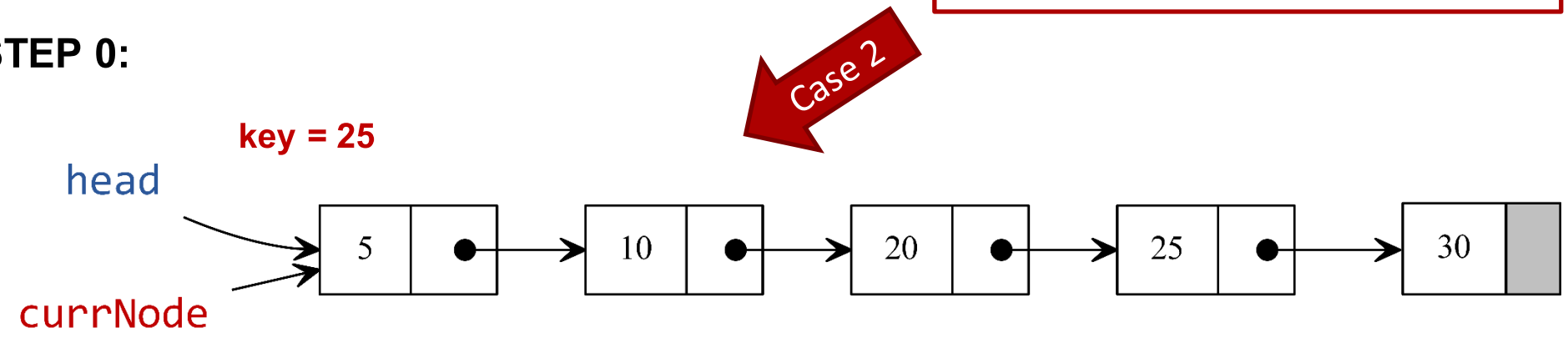
```
nodeT *findInList(nodeT *head, int key)
{
    nodeT* currNode = head;
    while (currNode && currNode->data < key) {
        currNode = currNode->next;
    }
    if (currNode && currNode->data == key)
        return currNode;
    return NULL;

}
```

A First Book of ANSI C, 4th Edition

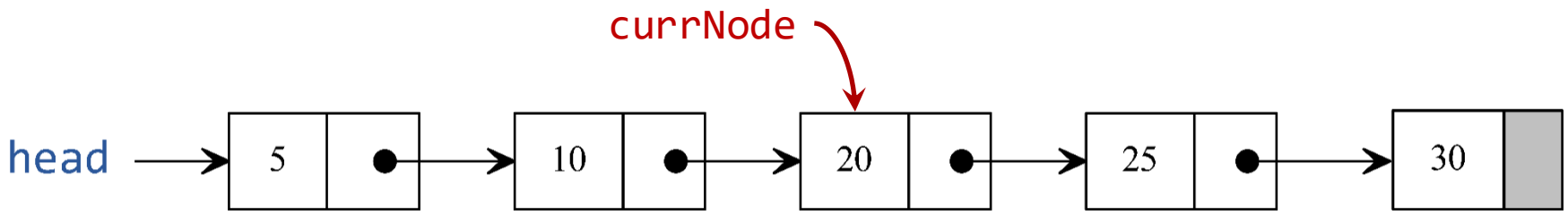
Delete from List

- Case 1: key อยู่ ตำแหน่ง head (Easy)
- Case 2: key อยู่ภายใน list

- **STEP 0:**



- **STEP 1:**



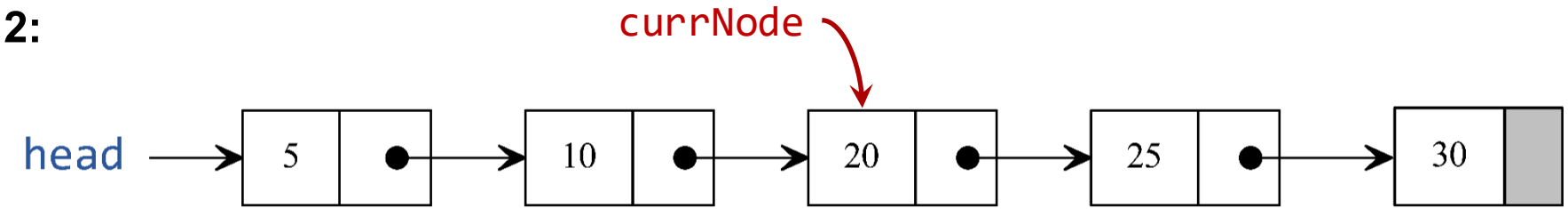

```
while ((currNode->next) &&
       (currNode->next->data < key))
       currNode = currNode->next;
```

A First Book of ANSI C, 4th Edition

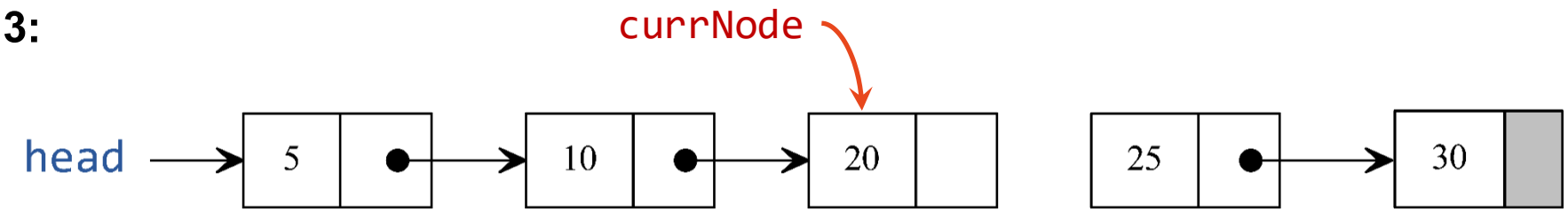
Delete from List [2]

```
if (!currNode->next || currNode->next->data != key) {
    printf("Key %d not found\n", key);
    return head;
}
```

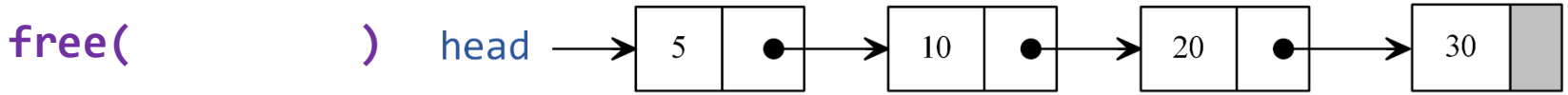
- **STEP 2:**



- **STEP 3:**



- **STEP 4:**

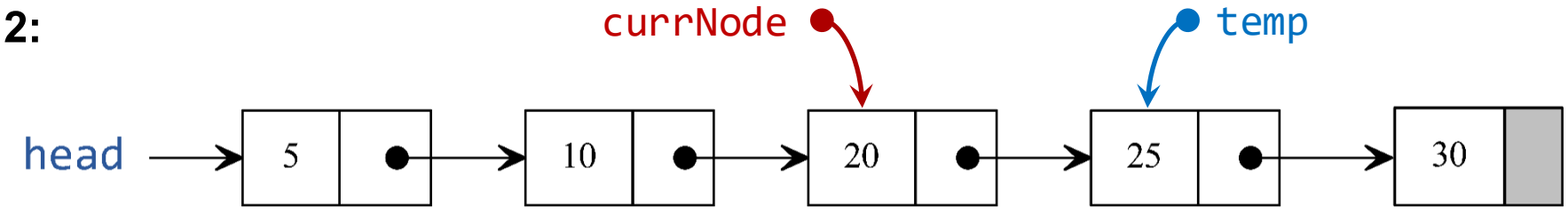


A First Book of ANSI C, 4th Edition

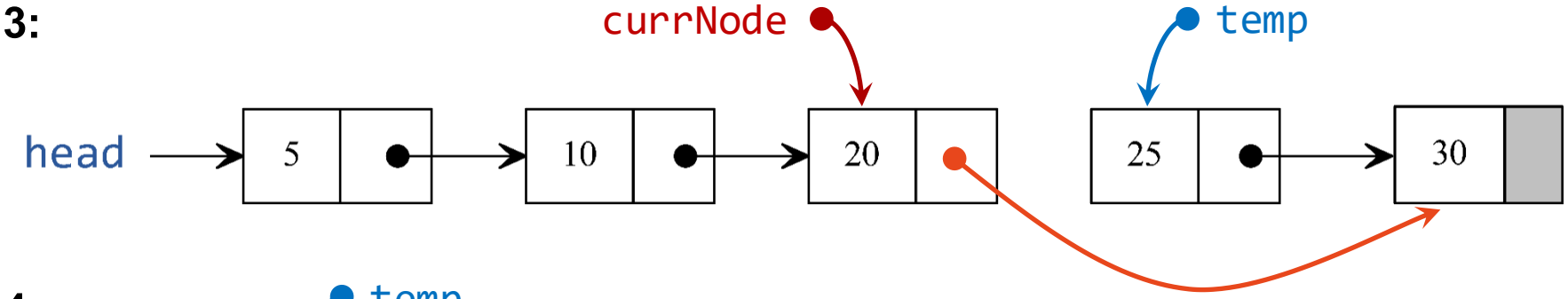
Delete from List [2: Key]

```
if (!currNode->next || currNode->next->data != key) {
    printf("Key %d not found\n", key);
    return head;
}
```

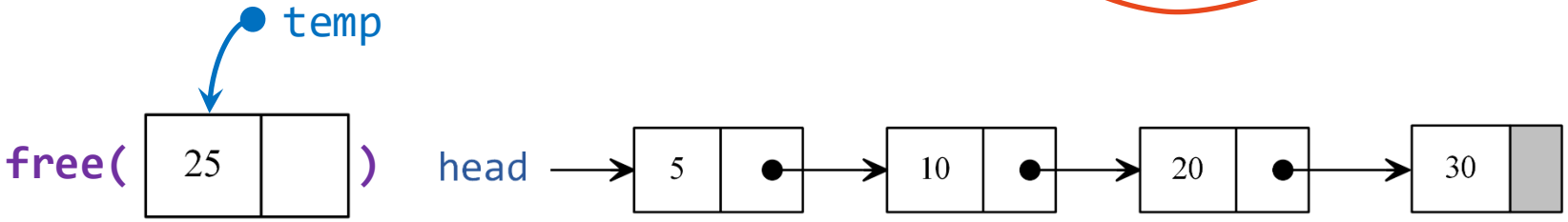
- **STEP 2:**



- **STEP 3:**



- **STEP 4:**



A First Book of ANSI C, 4th Edition

Delete from List [3: Key]

```
nodeT *deleteFromList(nodeT *head, int key)
{

    nodeT* temp = NULL;
    nodeT* currNode = head;

    if (head && head->data == key) {
        temp = head;
        currNode = head->next;
        free(temp);
        printf("Deleted: %d\n", key);
        return currNode;
    }
    while (currNode->next && currNode->next-> data < key) {
        currNode = currNode->next;
    }
    if (!currNode->next || currNode->next->data > key) {
        printf("Not found: %d\n", key);
    } else {
        temp = currNode->next;
        currNode->next = currNode->next->next;
        free(temp);
        printf("Deleted: %d\n", key);

    }
    return head;
}
```

A First Book of ANSI C, 4th Edition

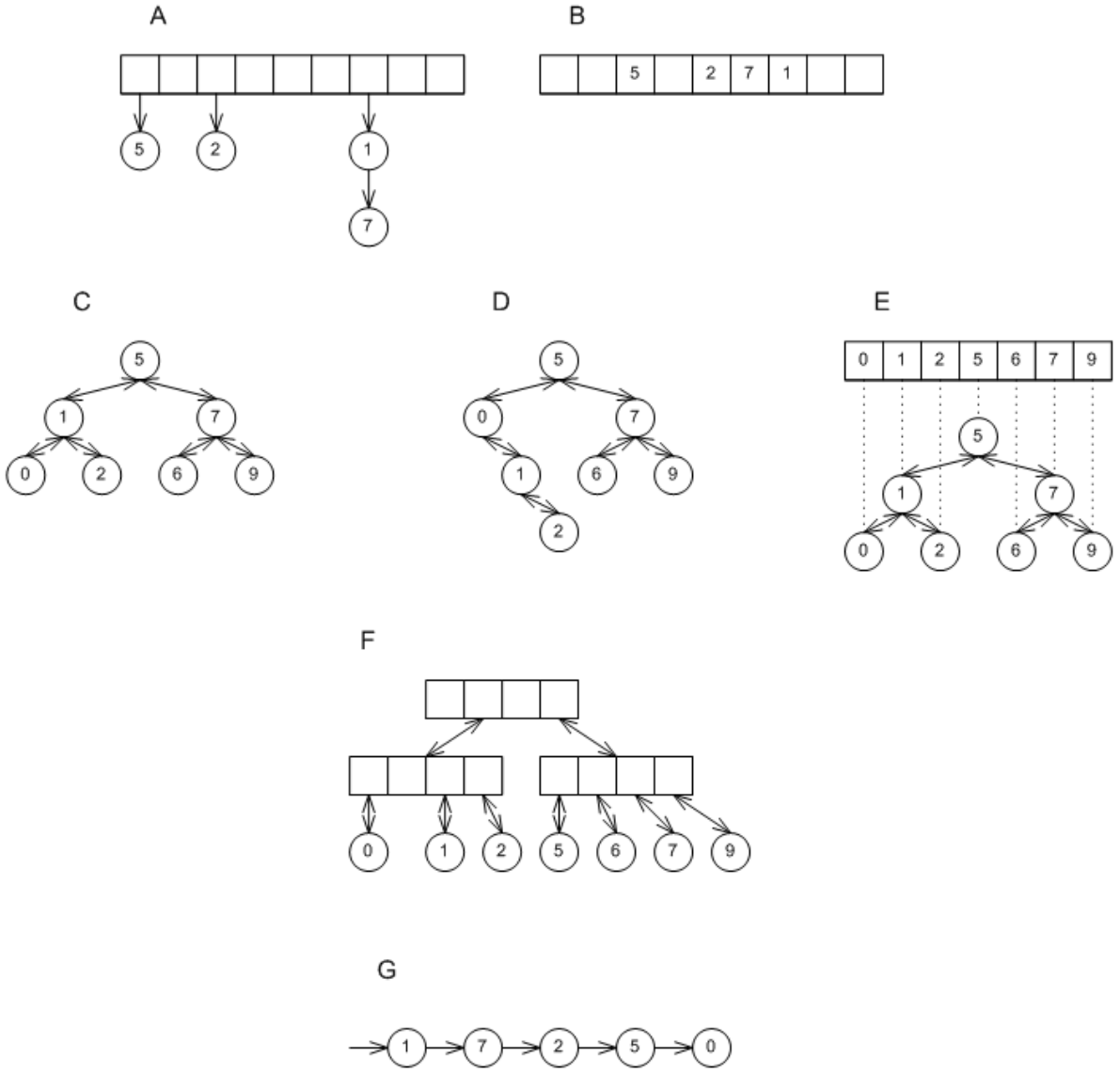
Array vs Linked List

- กรณีต้องการ **delete**
  - **Array:** หลังจาก **delete** ต้องขยับสมาชิกอื่น ๆ เพื่อมาแทนที่ว่าง
  - **Linked List:** ไม่มีผลกระทบหลังการ **delete**
- กรณีต้องการ **insert**
  - **Array:** ต้องขยับสมาชิกอื่น ๆ เพื่อสร้างที่ว่างให้ **insert** ได้
  - **Linked List: insert** ได้โดยไม่ต้องขยับสมาชิกอื่น ๆ

A First Book of ANSI C, 4th Edition

Other Types of Data Structures

- **Arrays**
- **Lists**
- Trees
- Binary trees
- B-trees
- Heaps
- Tries
- Multiway trees
- Space-partitioning trees
- Application-specific trees
- Graphs



A First Book of ANSI C, 4th Edition

Reference

- http://ocw.mit.edu/courses/electrical-engineering-and-computer-science/6-087-practical-programming-in-c-january-iap-2010/
- http://www.catb.org/esr/structure-packing/
- http://en.wikipedia.org/wiki/List_of_data_structures